# Engineering Design Document: Cloud-Native Microservices Architecture

## Overview

This document outlines the software architecture for our next-generation SaaS platform built using Python, React, and Kubernetes-based container orchestration.

## System Architecture

The backend API server is implemented using a microservices pattern with REST and GraphQL endpoints. Each service is containerized using Docker and deployed via Kubernetes with auto-scaling infrastructure managed through Terraform (Infrastructure as Code).

## Technology Stack

- Backend: Python, FastAPI, PostgreSQL database
- Frontend: React, TypeScript, Node.js
- Cloud Infrastructure: AWS Lambda, S3, EC2 with serverless compute
- CI/CD Pipeline: GitHub Actions with automated deployment to staging and production
- Monitoring: Observability stack with Prometheus and Grafana dashboards

## Machine Learning Components

Our data science team uses TensorFlow and PyTorch for model training. The pipeline ingests data via pandas and numpy preprocessing scripts, with the trained model exposed through a REST API endpoint. Embeddings are generated using a transformer-based architecture similar to BERT for natural language processing tasks.

## Cybersecurity Considerations

All API traffic is encrypted using TLS/SSL certificates. The security team conducts regular penetration testing and vulnerability scanning. A SIEM platform monitors for potential threats including phishing attempts and zero-day exploits. Firewall rules restrict access to internal microservices.

## Agile Development Process

The product roadmap is managed through quarterly OKRs and sprint planning sessions. Each two-week sprint includes a backlog grooming session, daily standups, and a sprint retrospective. User stories are tracked with acceptance criteria in our project management tool.

## DevOps Practices

The platform uses GitOps principles for deployment, with infrastructure changes tracked through version-controlled repositories. Code reviews are mandatory before merging to the main branch, and automated test coverage must exceed 80% for all new pull requests.